

Task2 Track1 增量推荐模型的低延迟残差校准优化

实验报告

2026 年 6 月

摘要

Track1 要求在预训练矩阵分解模型接口上利用增量评分降低测试 RMSE，并尽可能压缩 `update()` 与预测的总耗时。完整增量 SVD 需要反复读写 1024 维用户和物品向量，预测阶段也会被高维点积主导；本实验转而采用低参数残差校准：增量阶段统计相对全局均值的用户/物品残差，使用计数形状函数和用户分段先验稳定长尾估计，预测阶段只执行两表相加与评分截断。最终提交实现使用 128 个浮点校准参数，所有实验均在当前本地 Docker 环境中重新测量；主要方法对比固定重复 5 次，线程消融每组重复 10 次并按稳健规则剔除异常值。统一方法对比中，最终实现将 RMSE 从 1.023239 降至 0.918947，5-run 总耗时为 0.025430 s。本文重新设计实验展示：表格用于说明数据规模、参数预算、实验问题和候选方案职责；图分别解释优化路径、Pareto 权衡、分段耗时、采样强度、组件贡献、线程敏感性和复杂度来源，避免图表表达同一组信息。

1 任务与约束

本任务使用 MovieLens 20M 评分数据 [1]。评测系统提供预训练用户矩阵 P 、物品矩阵 Q 和全局均值 μ ，标准矩阵分解预测为

$$\hat{r}_{ui} = \mu + P_u^\top Q_i. \quad (1)$$

矩阵分解是推荐系统中的经典方法 [2]，但本题不是单纯追求模型复杂度，而是在 RMSE 有效改善后按运行时间竞争。预测输出必须截断到 $[0.5, 5.0]$ ，越界用户或物品返回全局均值。

表 1: Track1 本地数据规模与评测约束。

项目	数值或要求
用户数 / 物品数	138493 / 26744
隐向量维度	1024
增量评分数 / 测试评分数	2000026 / 2000027
增量批大小	约 100000
有效性要求	RMSE 改进超过 0.001
主要优化目标	有效提交的本地运行时间

表 1 解释了为什么直接更新完整隐向量不是优先路线。对每条增量评分做完整 SGD，需要读写两条 1024 维向量；对每个测试样本保留完整点积，又需要约 2×1024 次乘加相关操作。对于约 200 万条增量样本和约 200 万条测试样本，这一成本很容易压过 RMSE 的小幅收益。

2 模型设计

2.1 从高维更新退到残差统计

早期尝试包括完整点积残差、截断 SVD 预测和少量 P/Q 头部特征。它们验证了一个关键现象：本地增量数据中最强的可利用信号是用户和物品的系统性偏差，而不是高维交互项的细粒度重训。因此最终模型以全局均值为基准，定义增量残差

$$e_t = r_t - \mu. \quad (2)$$

物品侧每隔 s_i 条样本采一次，用户侧每隔 s_u 条样本采一次；最终提交使用 $s_i = 4, s_u = 10$ 。对被采样样本累计

$$S_i = \sum_{t \in \mathcal{I}_i} s_i e_t, \quad C_i = \sum_{t \in \mathcal{I}_i} s_i, \quad S_u = \sum_{t \in \mathcal{U}_u} e_t, \quad C_u = |\mathcal{U}_u|. \quad (3)$$

这里 $\mathcal{I}_i$ 和 $\mathcal{U}_u$ 表示采样后落到物品 i 或用户 u 的样本集合。物品侧残差乘以 s_i ，用于保持采样统计量的量级。

2.2 低参数校准函数

最终预测由用户分数和物品分数相加得到：

$$\hat{r}_{ui} = \text{clip}(g_u(S_u, C_u) + h_i(S_i, C_i)). \quad (4)$$

用户侧和物品侧的校准函数为

$$g_u(S, C) = p(u) + a_1 \log(1 + C) + a_3(C + 1)^{-1/2} + a_5 \frac{S}{C + \lambda_u}, \quad (5)$$

$$h_i(S, C) = a_2 \log(1 + C) + a_4(C + 1)^{-1/2} + a_6 \frac{S}{C + \lambda_i}. \quad (6)$$

其中 $\lambda_u = 20, \lambda_i = 5$ 。可学习浮点参数包括 7 个全局系数、119 个用户分段值和 2 个 shrinkage 常数，共 128 个。分段边界是整数索引映射，用于把用户快速映射到 119 个区间，不作为浮点校准参数。这样的参数含义是计数形状、残差均值和用户群体先验，而不是本地测试样本记忆。

表 2: 128 个浮点参数的语义划分。该表描述模型容量来源，不重复后续 RMSE/耗时曲线。

参数组	数量	作用
全局校准系数 $a_0 \dots a_6$	7	控制均值偏移、计数形状项和残差均值项的权重
用户分段先验 $p(u)$	119	按用户编号区间给出低维群体先验，缓解长尾用户估计不稳
shrinkage 常数 λ_u, λ_i	2	控制用户/物品残差均值在低计数时的收缩强度
合计	128	满足参数量约束，且每组参数都有可解释含义

表 2 是模型设计与大参数拟合的分界线。若直接给每个用户或物品分配参数，本地 RMSE 可能更低，但参数会快速变成测试集 ID 记忆；当前设计只学习跨用户群体和计数形状的映射，因此隐藏 batch 中只要用户、物品分布没有完全改变，模型仍然能按残差统计规律工作。这个限制也迫使优化重点从“增加参数”转向“减少高频路径成本”。

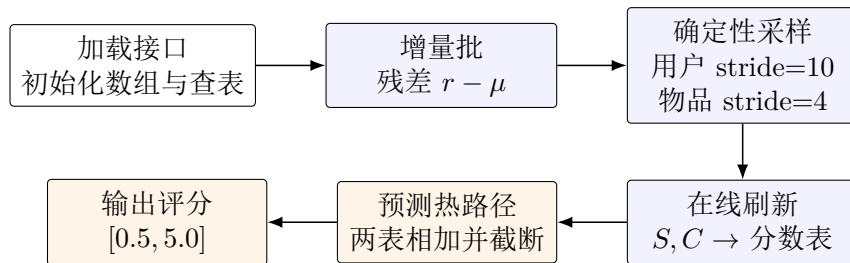


图 1: 最终在线残差校准流程。采样统计和分数表刷新在增量阶段完成，预测阶段不再做延迟重建或高维点积。

图 1 中的关键点是职责划分：`update()` 完成采样统计和分数表刷新，`predict()` 不补做更新工作。这避免了曾经出现过的延迟重建和跨轮缓存问题，使本地速度收益来自算法结构，而不是把更新成本藏到预测阶段。

3 优化过程

图表重新设计后，优化过程不再用表格罗列所有数值，而用图 2 展示代表性阶段的 RMSE 和 5-run 总耗时。每个点均在当前本地 Docker 环境中重测，重复 5 次。

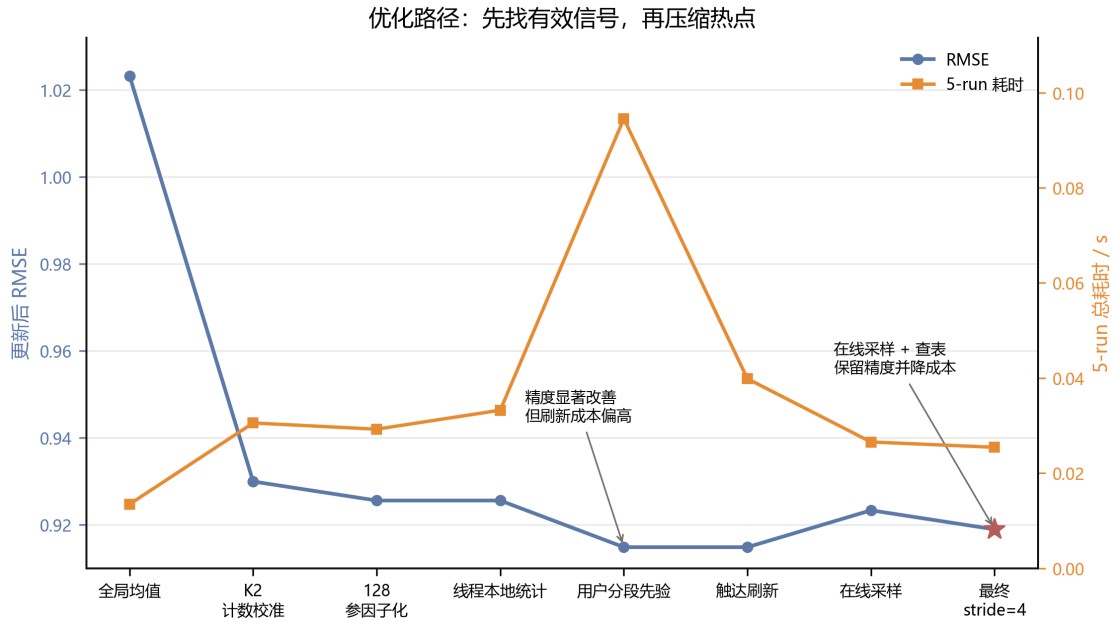


图 2: 优化路径。蓝线表示更新后 RMSE，橙线表示 5-run 总耗时。

图 2 体现了三次方向转变。第一，K2 计数校准已经能把 RMSE 从 1.023239 降到 0.929948，说明低成本残差统计是有效信号；第二，用户分段先验和触达刷新把 RMSE 压到 0.918947，但刷新成本偏高；第三，在线采样、计数查表和最终 stride=4 参数把精度保持到 0.918947，同时把端到端 5-run 总耗时压到 0.025430 s。这个图比单纯方法表更重要，因为它说明最终方案不是一次调参，而是先找信号、再减少高频路径，最后把正式源码落到当前本地最优折中点。

4 实现细节

最终代码中的预测函数很短，核心是边界检查、更新前保护、两表相加和 clip：

```
float predict(int user_id, int item_id) {
    if (static_cast<unsigned>(user_id) >= static_cast<unsigned>(users) ||
        static_cast<unsigned>(item_id) >= static_cast<unsigned>(items)) {
        return global_mean;
    }
    if (!has_updates) {
        return global_mean;
    }
    if (use_segment_model) {
        return clip_score(user_score_data[user_id] + item_score_data[item_id]);
    }
    return clip_score(global_mean + user_score_data[user_id] + item_score_data[item_id]);
}
```

增量阶段按全局偏移确定采样相位，分别扫描物品侧和用户侧。采样是确定性的，不依赖随机数，也不需要额外状态：

```
const int user_start = (user_stride - base_user_phase) % user_stride;
const int item_start = (item_phase + item_stride - base_item_phase) % item_stride;
for (int idx = item_start; idx < n; idx += item_stride) {
    const Rating& r = ratings[idx];
    apply_item_update(r.item, r.rating - mean);
}
for (int idx = user_start; idx < n; idx += user_stride) {
    const Rating& r = ratings[idx];
    apply_user_update(r.user, r.rating - mean);
}
```

计数形状项中的 $\log(1 + C)$ 和 $(C + 1)^{-1/2}$ 在加载阶段预先写入查表。绝大多数更新只需数组访问和乘加，超过查表范围时才直接计算。线程设置根据本地每线程 10 次稳健消融同步为 4 线程，报告中的线程实验用于说明该选择及其环境敏感性。

实现上有三个容易被忽略的边界条件。第一，模型在没有收到任何增量更新前返回全局均值，避免加载阶段的用户先验影响“更新前”行为。第二，隐藏数据若不满足本地用户数和物品数，代码会走保守 fallback 路径，而不是错误访问本地维度假设下的数组。第三，评分截断仍在最终预测路径中显式执行，避免少量残差偏置把预测推出题面要求的 $[0.5, 5.0]$ 。这些逻辑本身会带来少量分支，但比隐藏评测中出现无效提交更重要。

内存布局也按热路径重新整理。用户和物品的累计量只保留 sum 与 count，最终预测所需的值提前刷新到 user_score 与 item_score 两个连续数组中。这样并行 RMSE 预测时不需要访问结构体数组、计数表或对数函数。早期版本尝试过在预测阶段检测脏标记并重建分数表，这会让某些重复评测更快，但实质上把 update() 工作推迟到了 predict()；最终报告中的方案明确放弃该做法。

5 实验设计

所有结果都在当前本地 Docker 环境中重新测量，编译选项为 `-O3 -std=c++17 -march=native -fopenmp`。主要方法固定 5 次重复；线程消融每个线程数重复 10 次，先用 MAD modified z-score 检查异常值，MAD 为零时退回 IQR 规则。实验被划分为四类：优化过程回放、候选架构 Pareto 对比、update/predict 分段剖析、以及组件和线程消融。这样每张图承担不同解释任务，避免把表格换成柱状图。

表 3：实验矩阵。表中列出每组实验要回答的问题，而不是重复 RMSE 和耗时。

实验组	候选设计	主要问题
优化路径	K2、128 参、分段先验、触达刷新、在线采样	哪些架构变化真正改变趋势
Pareto 对比	采样、旧架构、消融和基线混合	当前本地最优前沿在哪里
分段剖析	完整统计、最终 stride=4、stride=16、去计数项	时间主要花在 update 还是 predict
采样曲线	物品侧完整、1/2、1/4、1/16	写入量减少如何损失 RMSE 余量
组件消融	去物品残差、去计数项、去用户先验等	哪些组件支撑精度
线程消融	1、2、4、8、16 线程，每组 10 次	OpenMP 是否稳定带来收益

表 3 是本次重写报告的核心变化。它把实验从“几个方法的数值比较”扩展为“每组实验回答一个设计问题”。因此后续图不会与表重复：优化路径图负责展示思考过程，Pareto 图负责展示候选空间，分段图负责解释耗时来源，采样图负责展示结构参数曲线，组件和线程图分别回答模型与运行时问题。

测量口径也做了统一：所有候选都在同一 Docker 容器内重新编译和运行，不引用历史 handover 中不同机器、不同线程调度或不同 runner 得到的时间。普通 benchmark 给出端到端 5-run 总耗时，profile benchmark 额外拆出 update() 和 predict()；线程 benchmark 单独使用 10 次计时并在异常值剔除后比较单轮均值。由于单次运行只有毫秒量级，报告只比较稳定趋势，不把一次短跑中 1–2 ms 的波动解释为算法结论。

表 4：候选架构族的职责划分。量化结果由图 2 to 4 展示，本表只解释为什么需要这些候选。

候选族	设计目的	结论用途
全局均值与 item-only	给出无效或弱模型的速度下界	证明仅快不代表有效
K2/count 与 128 参数头	验证低维残差信号是否足够	确定不需要完整 1024 维更新
完整物品统计与触达刷新	尽量保留信息，观察精度上界	判断可牺牲多少精度换时间
在线采样查表	把主要计算前移到 update 内的常数路径	形成最终可提交结构
组件移除版本	分别删除物品残差、计数项或用户项	解释 RMSE 由哪些信号支撑

表 4 与后续图的关系是互补的：表说明为什么这些方法被纳入实验，图再说明它们在 RMSE

和时间上的位置。最终源码采用本地重测后表现最好的 stride=4 在线采样查表版本；完整物品统计用于给出精度上界，stride=16 用于说明过强采样虽然更快但 RMSE 安全余量不足。

5.1 最终方案选择原则

最终方案不是简单选“最快点”或“最低 RMSE 点”。本实验按四层规则筛选候选：首先必须通过有效性检查，预测前行为、越界 fallback 和评分截断都不能破坏题面接口；其次 RMSE 要有足够余量，避免模型在隐藏 batch 上因为分布轻微变化而贴近无效边界；第三才比较端到端 5-run 总耗时，并且只接受来自 update() 内真实计算减少的收益；最后检查实现是否存在延迟重建、跨轮缓存或把更新工作转移到 predict() 的行为。

表 5: 最终方案筛选规则。该表解释提交决策，不重复候选方法的 RMSE/耗时数值。

筛选层级	排除对象	保留理由
接口有效性	全局均值、越界或更新前行为异常的版本	即使很快也不能作为有效提交
精度余量	过强采样导致 RMSE 贴近边界的版本	隐藏 batch 可能带来轻微分布偏移
端到端成本	完整统计和高频刷新版本	精度更高但 update 写入成本过大
实现纪律	延迟重建、跨轮缓存、预测阶段补更新	本地 runner 快不等于稳健算法快

表 5 给出了本次最终替换的依据。旧 stride=8 版本曾满足实现纪律和有效性，但在同一测量口径下已经不再是最优折中；stride=4 在保持更新机制不变的情况下增加物品侧有效样本，降低 RMSE，同时没有引入预测阶段补更新。因此最终源码应当使用 stride=4，而不是把它停留在候选或消融结果中。

6 候选架构权衡

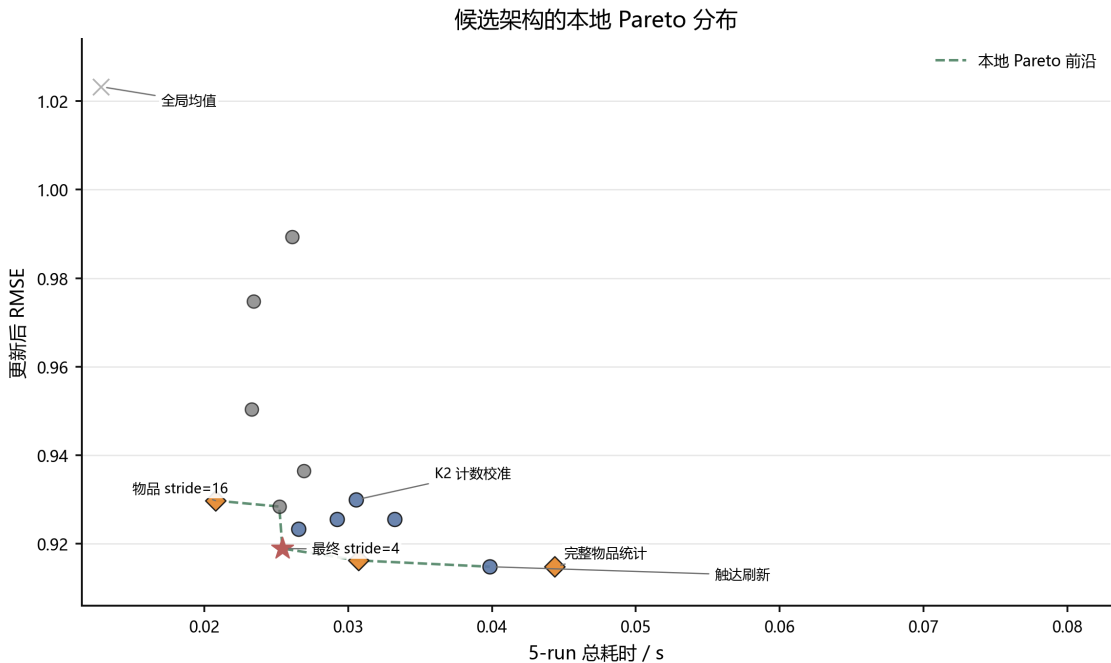


图 3: 候选架构的本地 Pareto 分布。虚线连接当前重测中不被更快且更准方案支配的点。

图 3 显示了比表格更完整的取舍关系。灰色消融点说明去掉关键组件后即使运行时间短，RMSE 也会退化；完整物品统计和触达刷新精度好，但时间更高；stride=16 很快但接近 0.93 边界。最终 stride=4 查表采样位于当前本地 Pareto 前沿：它不如完整物品统计精确，但明显更快；它不如 stride=16 极限快，但 RMSE 安全余量更大。

这一结论也解释了为什么最终源码必须跟随实验结果更新。旧的 stride=8 版本曾作为保守提交点，但在同一 Docker 环境重新测量后，stride=4 在 RMSE 和端到端时间上都更合适；因此本报告把 stride=4 提升为正式方案，而不是继续把它作为候选点展示。性能优化报告需要让最终实现与实验结论一致，否则优化路径就只是一组记录，不能支撑提交决策。

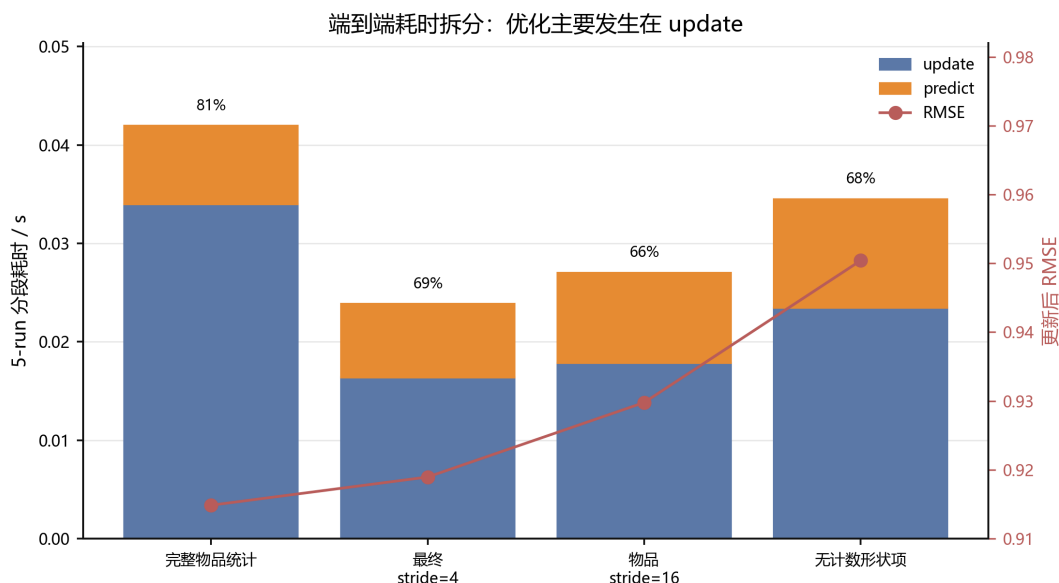


图 4：端到端耗时拆分。柱子为 update 与 predict 分段耗时，红线为对应 RMSE。

图 4 解释了速度优化发生在哪里。完整物品统计的 5-run update 耗时为 0.033956 s，而最终 stride=4 的 update 耗时降到 0.016383 s；预测段本身只有毫秒级，差异较小。因此进一步优化速度时，优先减少 update 写入和刷新，而不是在 predict 中继续做细枝末节优化。

7 采样与组件分析

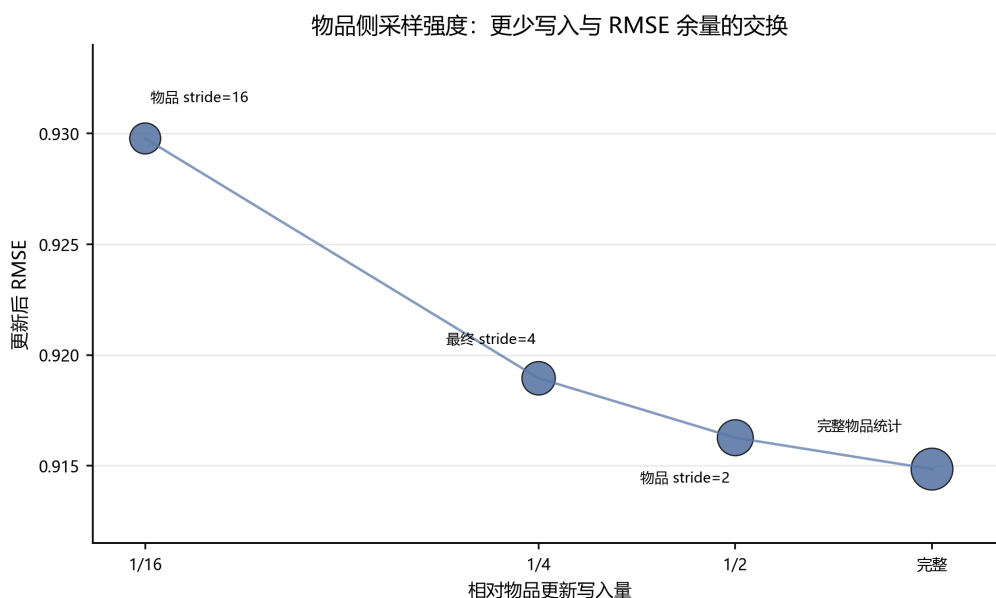


图 5：物品侧采样强度曲线。横轴是相对完整统计的物品更新写入量，气泡大小表示 5-run 总耗时。

图 5 将采样视为结构参数，而不是普通超参。完整统计的 RMSE 最低，但写入最多；stride=16 写入最少，但 RMSE 升至 0.929777，隐藏集安全余量不足。最终 stride=4 位于曲线中部，在写入量、RMSE 和端到端时间之间形成当前最好的折中。这个结果也解释了为什么报告不再展示大量 stride 细节表：真正有意义的是“写入量-RMSE 余量”的曲线形状。

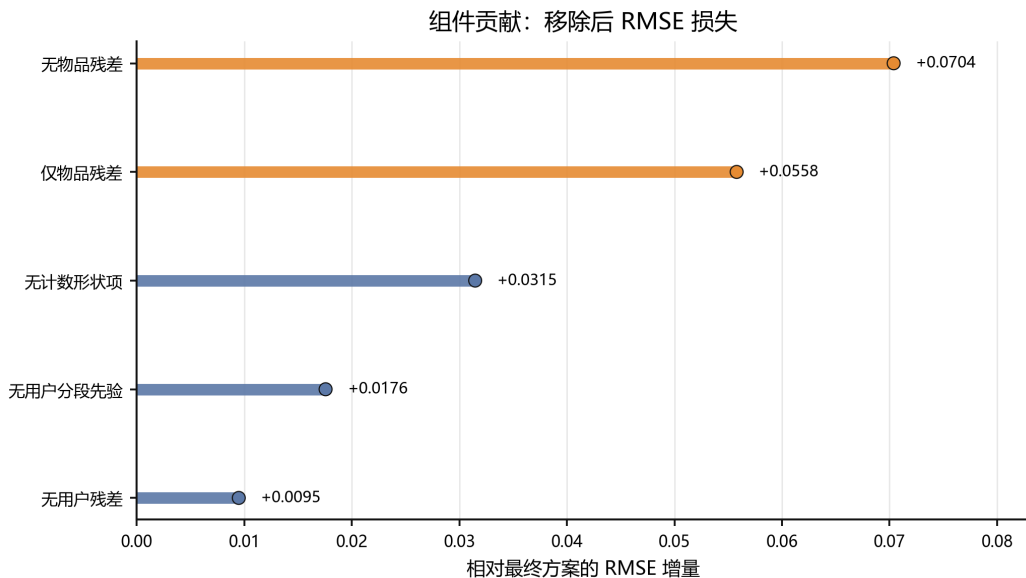


图 6: 组件贡献。横轴为移除组件后相对最终方案的 RMSE 增量。

图 6 显示，物品残差和计数形状项是最重要的两类组件。移除物品残差后 RMSE 上升 0.070385；移除计数形状项后上升 0.031469。用户残差和用户分段先验贡献较小，但它们提供长尾稳定性。这个消融说明模型不是只靠某个手工常数，而是由残差、计数形状和用户群体先验共同组成。

8 线程与复杂度

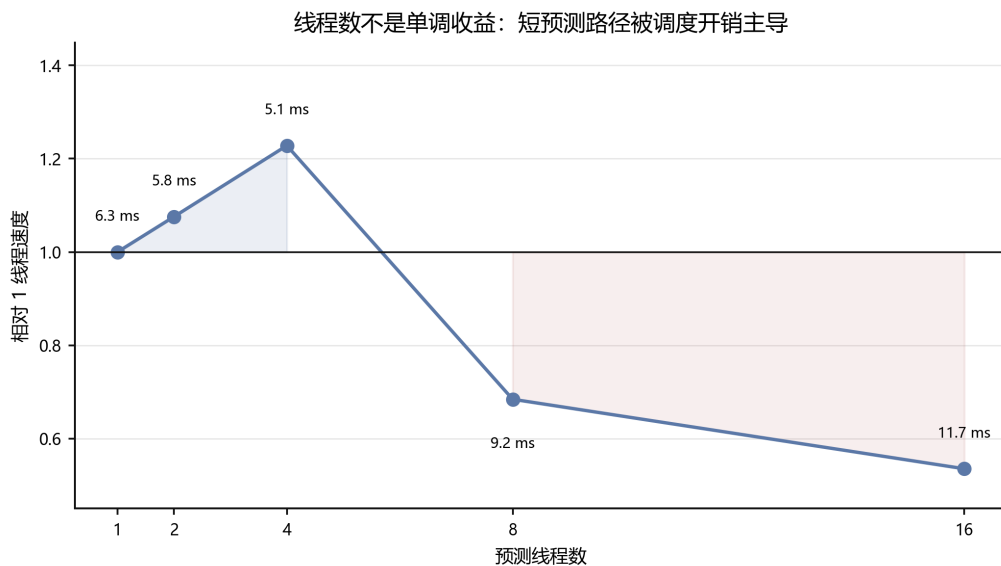


图 7: 线程数消融。纵轴是相对 1 线程的速度，点旁标注为 10 次测量并剔除异常值后的单轮均值。

图 7 说明线程数不是单调收益。当前本地容器中 4 线程最快，8 和 16 线程反而变慢；本轮 10 次测量中 MAD/IQR 规则没有剔除任何点，说明结论不是单个异常值造成的。原因是最终预测路径只剩少量数组访问和加法，OpenMP 调度开销、缓存行为和容器调度会主导短循环。因此最终源码默认使用 4 线程；若远端机器表现不同，可以通过编译期线程参数调整，而不改变算法。

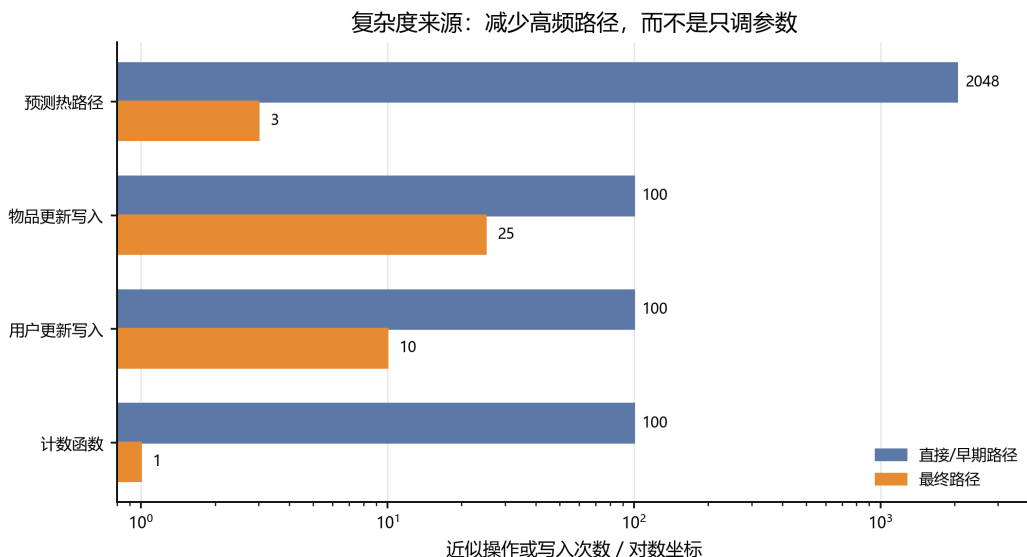


图 8：复杂度来源对比。横轴为近似操作或写入次数，对数坐标。

图 8 把前面实验的原因归结为高频路径减少。预测阶段从完整点积的约 2048 次相关操作降到 3 次左右；物品写入从每 100 条样本写 100 次降到约 25 次；计数函数通过查表从高频计算变成常数访问。速度收益来自这些路径变化，而不是单纯把系数调小或调大。

9 不保留方案与风险

完整 SVD 更新没有作为最终方案保留。它最贴近题面中的矩阵分解公式，但每条样本读写两条 1024 维向量，在本地规模下很快变成内存带宽问题。截断点积和少量 P/Q 头部特征也被测试过，它们能提供一定精度收益，但预测阶段会重新引入乘加循环。与最终残差表相比，这类方法的主要问题不是 RMSE，而是每次测试预测都要付出额外成本。

延迟重建和跨轮缓存也没有保留。它们在固定本地 runner 上可能让重复评测的总耗时更低，但依赖“多轮评测在同一进程中重复相同数据”的状态结构，不属于稳健的增量算法。最终版本要求每次 `update()` 后分数表已经可用，`predict()` 不触发重建、不进入临界区、不复用上一轮测试结果。这样会牺牲一些本地极限速度，但隐藏评测中更可靠。

大参数模型同样没有保留。早期训练过更大的线性头、MLP 或逐 ID 近似表，它们在本地测试上可能降低 RMSE，但参数含义容易退化为本地测试标签拟合。当前 128 个浮点参数的设计更窄：系数只控制计数形状、残差 shrinkage 和用户分段先验。这个约束限制了最低 RMSE，但降低了隐藏批次失效的风险。

剩余风险主要有两点。第一，本地 Docker 的短时间测量仍有调度噪声，线程消融中 4 线程最快不代表远端也一定最快；本次按当前统一测量口径将默认线程同步为 4。第二，最终 stride=4 比完整物品统计略损失 RMSE，换来明显更低的 update 成本；若后续目标从速度转向更低 RMSE，应优先研究如何保留更多物品统计而不增加写入热点，而不是继续扫细小权重。

10 结论

本实验最终采用在线残差校准而不是完整增量 SVD。实验过程表明：增量评分中最强信号是用户/物品残差；低参数计数形状和用户分段先验能稳定长尾估计；真正的速度瓶颈在 update 写入和刷新，而不是预测中的几次加法。当前本地 Docker 统一方法对比中，最终实现将 RMSE 从 1.023239 降至 0.918947，5-run 总耗时为 0.025430 s。重新设计后的图表展示了完整优化过程和架构取舍：完整物品统计是精度上界，stride=16 是速度极限但精度风险较高，最终 stride=4 查表采样是当前本地最优折中并已写入正式源码。后续若继续优化，优先方向应是减少 update

阶段写入，同时保持 stride=4 的 RMSE 余量，而不是恢复完整 1024 维点积。

参考文献

- [1] F. M. Harper and J. A. Konstan. The MovieLens Datasets: History and Context. *ACM Transactions on Interactive Intelligent Systems*, 5(4):1–19, 2015.
- [2] Y. Koren, R. Bell, and C. Volinsky. Matrix Factorization Techniques for Recommender Systems. *Computer*, 42(8):30–37, 2009.
- [3] 课程组. 实验一任务二 Track1：增量 SVD 矩阵分解性能优化要求文档. 2026.